EE-222 Data Structures and Algorithms

BS(CE)-2k21

LAB REPORT # 8

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	**5**	**5**	**5**	**15**
Obtained				

_____

Lab Instructor.
Kaynat Rana

Experiment # 8

Creating logical codes of C++ for implementing Stacks using Arrays and Linked List

Objective:

The objective of this session is to understand the various operations on stacks in C++.

1. Push
2. Pop
3. Traversal

Equipment Used:

Visual Studio C++

Procedure:

Stacks:

Stacks are the most important in data structures. The notation of a stack in computer science is the same as the notion of the Stack to which you are accustomed in everyday life. For example, a recursion program on which function call itself, but what happen when a function which is calling itself call another function. Such as a function 'A' call function 'B' as a recursion. So, the firstly function 'B' is call in 'A' and then function 'A' is work. So, this is a Stack. This is a Stack is First In Last Out data structure.

Insertions in Stack:

In Stacks, we know the array work, sometimes we need to modified it or add some element in it. For that purpose, we use insertion scheme. By the use of this scheme, we insert any element in Stacks using array. In Stack we maintain only one node which is called TOP. And Push terminology is used as insertions.

Deletion in Stack:

In the deletion process, the element of the Stack is deleted on the same node which is called TOP. In stacks, it's just deleting the index of the TOP element which is added at last. In Stacks Pop terminology is used as deletion.

Display of Stack:

In displaying section, the elements of Stacks are been display by using loops and variables as a reverse order. Such that, last element is display at on first and first element enters display at on last.

Static Approach Algorithm for Stack Operations:

1. Declare and initialize necessary variables, e.g., top = -1, MAXSIZE etc.

2.For push operation

If top = MAXSIZE - 1

print "stack overflow"

else top = top + 1;

Read item from user stack[top] = item

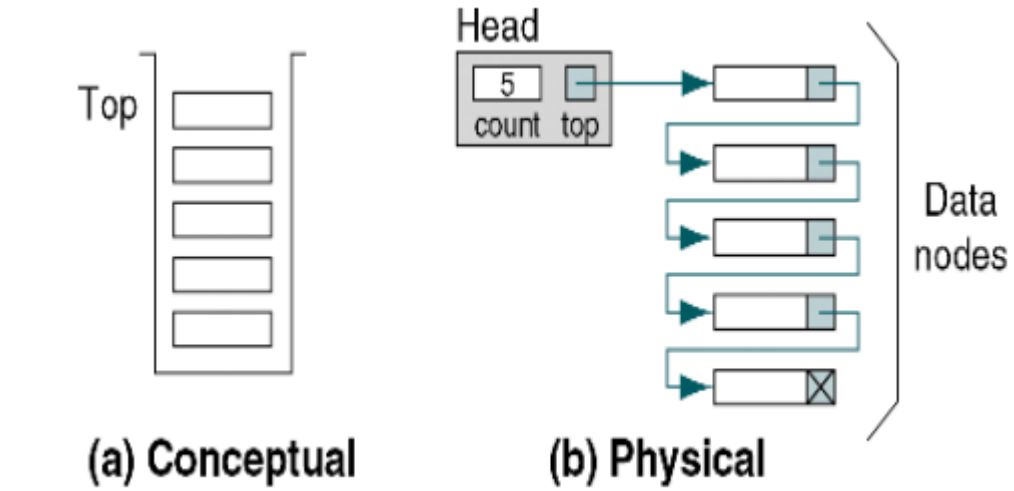
3. For next push operation, goto step 2.

4. For pop operation

If top = -1

print "Stack underflow"
Else item = stack[top]
top = top – 1
Display item
5. For next pop operation, goto step 4.
6. Stop

Dynamic Approach Algorithm for Stack Operations:



Following figure describes the dynamic approach for stack Implementation.

Step 1:

Declare a data structure for the nodes.e.g. the following struct could be used to create a list where each node holds a float –

```
structSNode
{
int data;
SNode *next;
};
```

Step 2:

Declare a pointer to serve as the stack top, e.g. SNode *top; Before you use this pointer, make sure it is initialized to NULL. Once you have done these 2 steps (i.e., declared a node data structure, and created a NULL top pointer, you have an empty stack. The next thing is to implement operations with the stack. Following algorithms describe the push and pop operations on stack.

Push Stack Design (Linked List)

• Create a new node (T)
• Store data in newly created node
• If stack is empty (top=NULL)
make new node as first node of stack and set top=T
• Else
Set the pointer of new node(T) to top
Update top with T.

Pop Stack Design (Linked List)

• If, top=NULL

Display stack is empty and exit.
• Else
Create a temporary stack node (temp)
Set Temp=top;
Set the top to previous node(top=top->next)
Delete temp node.

Tasks

Task 1(i):

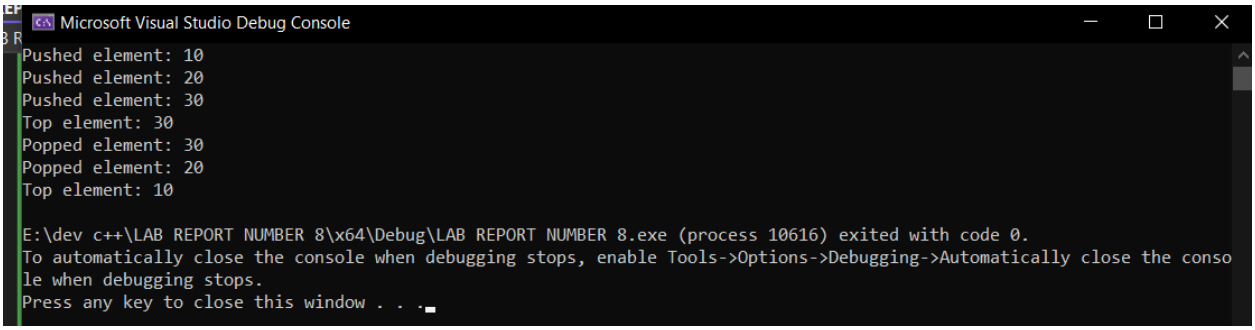
Create stack ADT in C++:

Using Static approach

CODE:

```cpp
#include <iostream>
#define MAX_SIZE 100
using namespace std;
class Stack {
private:
    int arr[MAX_SIZE];
    int top;
public:
    Stack()
    {
        top = -1;
    }
    void push(int data)
    {
        if (top >= MAX_SIZE - 1)
        {
            cout << "Stack overflow! Cannot push element." << er
            return;
        }
        arr[++top] = data;
        cout << "Pushed element: " << data << endl;
    }
    int pop()
    {
        if (top < 0)
        {
            cout << "Stack underflow! Cannot pop element." <<enc
            return -1;
        }
        int data = arr[top--];
        cout << "Popped element: " << data << endl;
        return data;
    }
    int peek()
    {
        if (top < 0)
        {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        return arr[top];
    }
    bool isEmpty()
    {
        return top < 0;
    }
};
int main()
{
    Stack stack;
    stack.push(10);
    stack.push(20);
    stack.push(30);
    cout << "Top element: " << stack.peek() <<endl;
    stack.pop();
    stack.pop();
    cout << "Top element: " <<stack.peek() << endl;
    return 0;
}
```

OUTPUT:



Task 1(ii):

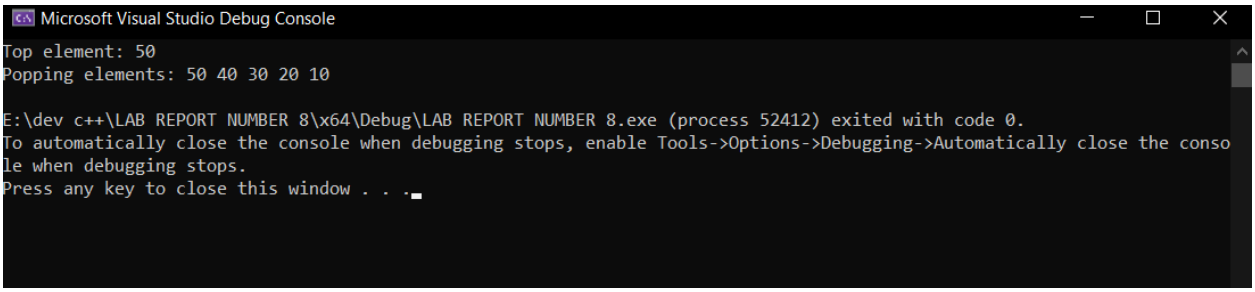
Create stack ADT in C++:

Using Dynamic approach

CODE:

```cpp
#include <iostream>
using namespace std;
class Stack
{
private:
    int* stackArray;
    int capacity;
    int top;
public:
    Stack(int size)
    {
        capacity = size;
        stackArray = new int[capacity];
        top = -1;
    }
    ~Stack()
    {
        delete[] stackArray;
    }
    bool isEmpty()
    {
        return (top == -1);
    }
    bool isFull()
    {
        return (top == capacity - 1);
    }
    void push(int element)
    {
        if (isFull())
        {
            cout << "Stack overflow! Cannot push element " << element << endl;
            return;
        }
        stackArray[++top] = element;
    }
    int pop()
    {
        if (isEmpty())
        {
            cout << "Stack underflow! Cannot pop element from an empty stack." << endl;
            return -1;
        }
        return stackArray[top--];
    }
    int peek()
    {
        if (isEmpty())
        {
            cout << "Stack is empty." <<endl;
            return -1;
        }
        return stackArray[top];
    }
};
int main()
{
    Stack myStack(5);
    myStack.push(10);
    myStack.push(20);
    myStack.push(30);
    myStack.push(40);
    myStack.push(50);
    cout << "Top element: " << myStack.peek() << endl;
    cout << "Popping elements: ";
    while (!myStack.isEmpty())
    {
        cout << myStack.pop() << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

OUTPUT:



Conclusion:

In conclusion, implementing stacks using arrays and linked lists in C++ involves writing logical code that allows for efficient insertion and deletion of elements in a Last-In-First-Out (LIFO) data structure.